Table des matières

Cours 05 — CTE et récursivité 1
Objectifs 1
Théorie 1
Points clés à retenir 9
Exercices 9

Cours 05 — CTE et récursivité

Module 5/12 — Formation SQL avancé

Comment lire ce module — Au Cours 04, les requêtes ont commencé à enfler, avec des sous-requêtes imbriquées et des fenêtres répétées. Ce module apporte l'outil qui remet de l'ordre, la CTE (`WITH`), puis le pousse jusqu'à sa forme la plus spectaculaire, la CTE récursive, qui sait générer une série de valeurs et remonter une hiérarchie. La théorie est ici ; les deux laboratoires (`Labo_05_OLAP_SQL.md` et `Labo_05_OLTP_SQL.md`) vous font tout taper.

Objectifs

À la fin de ce module, vous saurez :

- expliquer ce qu'est une CTE (`WITH`) et pourquoi elle rend une requête lisible ;
- transformer une sous-requête imbriquée en un pipeline de CTE chaînées ;
- choisir en connaissance de cause entre CTE, sous-requête et vue ;
- lire et écrire une CTE récursive, avec ses trois morceaux : l'ancre, `UNION ALL`, la partie récursive ;
- générer une série de valeurs sans table (nombres, dates) ;
- remonter une hiérarchie et calculer un niveau et un chemin ;
- déjouer les deux pièges : la récursion sans borne et les fausses idées sur la performance des CTE.

Théorie

1. Le problème : des requêtes qui deviennent illisibles

© 2026 iAlexMG. Tous droits réservés.

Labo : `Labo_05_OLTP_SQL.md` et `Labo_05_OLAP_SQL.md`, partie 1.

Souvenez-vous de la dernière requête du Cours 04 : les ventes par mois, avec la variation d'un mois à l'autre. Pour obtenir le total mensuel avant d'appliquer le `LAG`, on avait dû enfouir un `SELECT` dans un autre :

```
SELECT annee, mois, ventes,
       LAG(ventes) OVER (ORDER BY annee, mois)          AS ventes_mois_prec,
       ventes - LAG(ventes) OVER (ORDER BY annee, mois) AS variation
FROM (
    SELECT d.CalendarYear AS annee, d.MonthNumberOfYear AS mois,
           SUM(f.SalesAmount) AS ventes
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
    GROUP BY d.CalendarYear, d.MonthNumberOfYear
) AS mensuel
ORDER BY annee, mois;
```

Ça marche, mais ça se lit de l'intérieur vers l'extérieur : pour comprendre, l'œil doit plonger dans la parenthèse, en ressortir, puis relire le tout. Ajoutez un deuxième niveau et la requête devient un labyrinthe. Il y a mieux.

2. `WITH` : nommer un résultat intermédiaire

Labo : `Labo_05_OLTP_SQL.md` et `Labo_05_OLAP_SQL.md`, partie 1.

Une **CTE** (*Common Table Expression*, ou expression de table commune) est un résultat intermédiaire qu'on nomme, en tête de requête, avec le mot-clé `WITH`. On l'utilise ensuite comme si c'était une table, sauf qu'elle n'existe que le temps de la requête.

La même requête qu'en section 1, réécrite avec une CTE :

```
WITH mensuel AS (
    SELECT d.CalendarYear AS annee, d.MonthNumberOfYear AS mois,
           SUM(f.SalesAmount) AS ventes
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
    GROUP BY d.CalendarYear, d.MonthNumberOfYear
)
SELECT annee, mois, ventes,
       LAG(ventes) OVER (ORDER BY annee, mois)          AS ventes_mois_prec,
       ventes - LAG(ventes) OVER (ORDER BY annee, mois) AS variation
FROM mensuel
ORDER BY annee, mois;
```


© 2026 iAlexMG. Tous droits réservés.

Le résultat est identique, mais la lecture est devenue linéaire, de haut en bas : d'abord je calcule `mensuel`, ensuite je m'en sers. La sous-requête a un nom, l'intention saute aux yeux.

L'image à retenir — Une CTE, c'est une étape de travail qu'on pose sur l'établi et qu'on étiquette. La requête finale se contente d'assembler les étapes. Le `WITH` ne crée aucune table durable : rien n'est stocké, rien à nettoyer.

3. Chaîner les CTE : un pipeline lisible

Labo : `Labo_05_OLTP_SQL.md` et `Labo_05_OLAP_SQL.md`, sous-partie 2.1.

Le vrai gain apparaît quand on enchaîne plusieurs CTE : chacune part de la précédente, comme les étapes d'une chaîne de montage. On les sépare par une virgule, avec un seul `WITH` en tête.

Les mois au-dessus de la moyenne mensuelle de l'entreprise, en deux étapes claires :

```sql
WITH mensuel AS (                       -- etape 1 : le total de chaque mois
    SELECT d.CalendarYear AS annee, d.MonthNumberOfYear AS mois,
           SUM(f.SalesAmount) AS ventes
    FROM dbo.FactInternetSales AS f
    JOIN dbo.DimDate           AS d ON d.DateKey = f.OrderDateKey
    GROUP BY d.CalendarYear, d.MonthNumberOfYear
),
moyenne AS (                            -- etape 2 : la moyenne de tous ces mois
    SELECT AVG(ventes) AS ventes_moyens FROM mensuel
)
SELECT m.annee, m.mois, m.ventes        -- etape 3 : on ne garde que les mois au-dessus
FROM mensuel AS m
CROSS JOIN moyenne
WHERE m.ventes > moyenne.ventes_moyens
ORDER BY m.ventes DESC;
```

Chaque CTE fait une chose et porte un nom qui la décrit. Pour déboguer, il suffit de remplacer le `SELECT` final par `SELECT * FROM mensuel` (ou `FROM moyenne`) et d'inspecter l'étape voulue, ce qu'on ne peut pas faire aussi proprement avec des sous-requêtes empilées.

4. CTE, sous-requête ou vue ?

Labo : `Labo_05_OLTP_SQL.md` et `Labo_05_OLAP_SQL.md`, partie 1.

Trois façons de nommer ou de réutiliser un résultat. Elles ne jouent pas dans la même cour :

© 2026 iAlexMG. Tous droits réservés.

Outil	Portée	Persiste ?	Quand la choisir
Sous-requête	l'endroit exact où elle est écrite	non	un calcul court, utilisé une seule fois
CTE (`WITH`)	toute la requête qui suit	non	rendre une requête lisible ; réutiliser un résultat plusieurs fois dans la même requête ; la récursivité
Vue (`CREATE VIEW`)	toutes les requêtes futures	oui, c'est un objet enregistré	une logique partagée entre plusieurs requêtes, plusieurs jours

En un mot : la sous-requête est jetable et locale, la CTE structure une requête, la vue est un objet réutilisable rangé dans la base.

Rappel — Créer, remplacer et supprimer une vue sont présentés au Cours 03, section 7.

5. La récursivité : répéter jusqu'à une condition

Labo : `Labo_05_OLTP_SQL.md` et `Labo_05_OLAP_SQL.md`, sous-parties 2.2 et 2.3.

Une CTE **récursive** est une CTE qui se réfère à elle-même. C'est l'outil pour tout ce qui se déroule de proche en proche : générer 1, 2, 3, 4 ; remonter une chaîne de commandement ; parcourir un arbre de catégories. Elle a trois morceaux, écrits dans cet ordre :

Morceau	Ce qu'il fait	Ce qu'il doit contenir
L'ancre	le point de départ, une ou plusieurs lignes	une requête ordinaire, sans référence à la CTE
`UNION ALL`	empile les couches successives	rien à décider, sauf le choix de `ALL` (section 8)
La partie récursive	repart du résultat précédent et ajoute la couche suivante	une référence au nom de la CTE, et une condition d'arrêt

Le moteur exécute l'ancre une fois, puis relance la partie récursive encore et encore, chaque tour repartant des lignes que le tour précédent vient de produire, jusqu'à ce que la partie récursive ne renvoie plus rien. C'est cette condition d'arrêt qui empêche l'infini (section 8).

Le squelette, en une chaîne de flèches : ancre → `UNION ALL` → partie récursive → nouvelle couche → retour à la partie récursive, et ainsi de suite jusqu'à la couche vide.

Portabilité — Nos deux moteurs exigent le mot-clé `RECURSIVE` (`WITH RECURSIVE …`), et DuckDB le dit d'ailleurs en toutes lettres quand on l'oublie : « Circular reference to CTE, use WITH RECURSIVE to use recursive CTEs ». Certains moteurs s'en passent, dont Snowflake d'après sa documentation. Écrivez-le toujours : il est obligatoire là où vous travaillez, et toléré ailleurs.

6. Premier cas : générer une série sans table

© 2026 iAlexMG. Tous droits réservés.

Labo : `Labo_05_OLTP_SQL.md` et `Labo_05_OLAP_SQL.md`, sous-partie 2.2.

La récursivité la plus simple ne touche aucune table : elle fabrique des valeurs. Les douze premiers entiers :

```sql
WITH RECURSIVE nombres AS (
    SELECT 1 AS n                                -- ancre : on part de 1
    UNION ALL
    SELECT n + 1 FROM nombres WHERE n < 12       -- recursion : +1, tant qu'on n'a pas 12
)
SELECT n FROM nombres;
```

L'ancre pose `1`. Le tour suivant lit ce `1` et produit `2` ; le suivant, `3` ; et ainsi de suite. Au tour où `n` vaut `12`, la condition `n < 12` est fausse, la récursion ne renvoie plus rien, ça s'arrête. Résultat : une colonne de 1 à 12.

L'usage principal, c'est de combler des trous. Un rapport de ventes par mois saute les mois sans aucune vente, parce qu'il n'y a pas de ligne à agréger. En générant les 12 mois d'abord, puis en y raccrochant les ventes par un `LEFT JOIN`, les mois vides réapparaissent à zéro. On le fait au labo.

Portabilité — Générer une série est si courant que chaque moteur offre un raccourci. Nos deux terrains ont le même, `generate_series(1, 12)`, et DuckDB ajoute `range(1, 13)`, dont la borne haute est exclue. Snowflake écrit autrement, d'après sa documentation : `TABLE(GENERATOR(ROWCOUNT => 12))` fabrique douze lignes sans aucune colonne, pas douze nombres, et c'est une fonction à part qui les numérote, `ROW_NUMBER()` pour une suite sans trou, `SEQ4()` qui peut en laisser. La CTE récursive, elle, marche partout, et c'est pourquoi on l'apprend d'abord : le jour où le raccourci du moteur vous manque, elle reste.

7. Deuxième cas : parcourir une hiérarchie

Labo : `Labo_05_OLTP_SQL.md` et `Labo_05_OLAP_SQL.md`, sous-partie 2.3.

C'est l'usage vedette, celui qu'on vous demandera en entretien. Dans l'entrepôt, `DimEmployee` se réfère à elle-même : chaque employé porte un `ParentEmployeeKey` qui pointe vers son manager, lui-même une ligne de `DimEmployee`. Une table qui se pointe elle-même, c'est un arbre, et un arbre se parcourt par récursion.

```sql
WITH RECURSIVE organigramme AS (
    -- ANCRE : le grand patron, celui qui n'a pas de manager
    SELECT EmployeeKey, ParentEmployeeKey,
           FirstName || ' ' || LastName AS employe,
           1 AS niveau
    FROM dbo.DimEmployee
    WHERE ParentEmployeeKey IS NULL
    UNION ALL
    -- RECURSION : les employes dont le manager est DEJA dans l'arbre
    SELECT e.EmployeeKey, e.ParentEmployeeKey,
           e.FirstName || ' ' || e.LastName,
           o.niveau + 1
    FROM dbo.DimEmployee AS e
```


© 2026 iAlexMG. Tous droits réservés.

```
    JOIN organigramme    AS o ON e.ParentEmployeeKey = o.EmployeeKey
)
SELECT niveau, employe
FROM organigramme
ORDER BY niveau, employe;
```

L'ancre attrape la racine (le PDG, `ParentEmployeeKey IS NULL`) au niveau 1. Chaque tour de récursion trouve les employés rattachés aux personnes déjà placées, et leur donne le niveau du parent plus un. On déroule ainsi tout l'organigramme, étage par étage.

On peut aussi construire le chemin complet en concaténant à chaque étage, ce qui est pratique pour afficher `PDG > Directeur ventes > Représentant` :

```
-- dans l'ancre :      ... FirstName || ' ' || LastName AS chemin
-- dans la recursion : ... o.chemin || ' > ' || e.FirstName || ' ' || e.LastName
```

Rappel — La concaténation `||`, et le `NULL` qui efface tout le résultat, sont présentés au Cours 02, section 5.

8. Deux pièges : la boucle sans fin et le mythe de la performance

Labo : `Labo_05_OLTP_SQL.md`, sous-partie 2.3.

Piège 1, la récursion sans borne. Si la partie récursive n'a pas de condition d'arrêt, ou une condition qui n'est jamais atteinte, la CTE tourne à l'infini. Sur ce point, aucun de nos deux moteurs ne vous protège, et c'est important de le savoir avant d'écrire votre première récursion sérieuse. Retirez le `WHERE n < 12` de l'exemple de la section 6 et lancez la requête sur DuckDB : elle ne rend rien, ne lève rien, et tourne toujours trois minutes plus tard. PostgreSQL se comporte de la même façon, jusqu'à épuiser la mémoire. Certaines plateformes posent une limite de profondeur et coupent avec une erreur, dont Snowflake, d'après sa documentation, avec une centaine de tours réglables par `MAX_RECURSIONS`. Vous ne travaillez pas dessus.

Règle d'or, donc : toute CTE récursive doit contenir une condition qui finira par être fausse, un `WHERE n < …`, ou le simple fait qu'un arbre a un bas. Et quand l'arbre vient de données qu'on ne maîtrise pas, on ajoute une borne de sécurité, `AND niveau < 10`, qui ne sert jamais sauf le jour où elle sauve la séance.

PostgreSQL offre pour ce cas précis un outil que l'entrepôt n'a pas, et c'est l'un des rares écarts de ce module : la clause `CYCLE`, qui détecte la boucle au lieu de la borner.

```
WITH RECURSIVE parcours (depart, courant) AS (
    SELECT 1, 1
    UNION ALL
    SELECT p.depart, a.vers FROM parcours AS p JOIN arete AS a ON a.de = p.courant
) CYCLE courant SET boucle USING trace
SELECT courant, boucle FROM parcours;
```

Sur un graphe qui revient sur lui-même, 1 vers 2, 2 vers 3, 3 vers 1, cette requête rend quatre lignes et s'arrête : le moteur trace le chemin parcouru, reconnaît qu'il repasse par un sommet déjà vu, marque la ligne d'un `true` dans la colonne `boucle` et cesse de descendre. C'est exactement ce qu'on code à la main quand on traîne un tableau de clés visitées, en une clause plutôt qu'en dix lignes. DuckDB ne la connaît pas et répond « syntax error at or near CYCLE » : sur l'entrepôt, la détection de cycle reste à écrire soi-même.

© 2026 iAlexMG. Tous droits réservés.

C'est aussi la raison du `UNION ALL` plutôt que du `UNION`. Le `ALL` empile sans dédoublonner, ce qui est ce qu'on veut, et surtout ce qui coûte le moins : un `UNION` simple obligerait le moteur à comparer chaque nouvelle ligne à toutes les précédentes, à chaque tour.

Piège 2, croire que la CTE est plus lente ou plus rapide. Une CTE est d'abord une aide à la lecture ; ce n'est pas un levier de performance. Deux nuances de moteur à connaître :

- Sur PostgreSQL avant la version 12, une CTE était matérialisée de force, calculée puis stockée dans un coin, ce qui pouvait bloquer des optimisations. Depuis la 12, la règle est plus fine qu'on ne le dit souvent, et elle vaut d'être connue exactement : une CTE citée **une seule fois** est intégrée librement, une CTE citée **deux fois ou plus** reste matérialisée par défaut. Les deux mots-clés permettent de forcer l'un ou l'autre, `AS MATERIALIZED` et `AS NOT MATERIALIZED`. Le plan le montre sans ambiguïté : une CTE lue deux fois y apparaît comme `CTE Scan`, et la même déclarée `NOT MATERIALIZED` disparaît du plan, fondue dans la requête qui la lit.
- DuckDB fait de même : il traite la CTE comme une sous-requête nommée et optimise à travers, et il accepte lui aussi `AS MATERIALIZED` quand vous voulez lui imposer le calcul en une fois. C'est utile dans un seul cas, quand la CTE est coûteuse et lue plusieurs fois dans la même requête.

Autrement dit : écrivez des CTE pour la clarté. La performance se joue ailleurs, dans les index et les plans d'exécution, au Cours 10.

9. Sur les deux terrains : OLTP et OLAP

Labo : `Labo_05_OLTP_SQL.md`, partie 3.

Comme les fonctions de fenêtrage, les CTE sont du SQL standard et se comportent à l'identique sur un moteur transactionnel et sur un entrepôt. Le tableau ci-dessous est même remarquable par ce qu'il ne contient pas.

Ce qu'on veut	Voie OLTP (PostgreSQL, schéma normalisé)	Voie OLAP (DuckDB, étoile)
Déclarer une CTE récursive	`WITH RECURSIVE` obligatoire	obligatoire aussi
Générer une série d'entiers	`generate_series(1, 12)`, ou la CTE récursive	`generate_series(1, 12)` ou `range(1, 13)`, ou la CTE récursive
Se protéger d'une boucle	à borner soi-même, le moteur ne coupe pas	à borner soi-même aussi, le moteur ne coupe pas davantage
Forcer le calcul d'une CTE	`AS MATERIALIZED`	`AS MATERIALIZED`
La syntaxe du pipeline	`WITH a AS (…), b AS (…) SELECT …`	identique, au caractère près
Un arbre à parcourir	`production.billofmaterials`, la nomenclature	`DimEmployee`, l'organigramme mis à plat
Écarter les lignes périmées	à filtrer soi-même (`enddate IS NULL`)	à filtrer soi-même aussi (`Status = 'Current'`), et le piège y est pire

© 2026 iAlexMG. Tous droits réservés.

Les cinq premières lignes disent la même chose des deux côtés, et c'est le constat du cours : sur les CTE, il n'y a pratiquement pas de dialecte à apprendre. Ce cours est celui où le double chemin rapporte le moins d'écarts de syntaxe, et c'est une information en soi. Les deux dernières lignes, elles, sont d'une autre nature : elles ne parlent pas du moteur mais du modèle, et c'est là que tout se joue.

Le premier de ces deux écarts demande un mot, et il n'est pas celui qu'on attend. Les deux côtés portent des dates de validité ; ce qui change, c'est où elles se trouvent. La base transactionnelle vous donne l'arbre tel que l'application le tient, avec ses dates sur le lien, parce que c'est ce dont l'application a besoin pour fonctionner. L'entrepôt, lui, vous donne une colonne parent posée à plat, ce qui est plus simple à parcourir, mais il garde son historique dans la dimension elle-même : `DimEmployee` porte `StartDate`, `EndDate` et `Status`, et une nouvelle ligne naît à chaque changement. C'est une dimension à évolution lente, du type que la modélisation dimensionnelle appelle « type 2 ».

Les chiffres le disent mieux qu'une phrase. La table compte 296 lignes pour 290 personnes : six lignes sont des versions périmées, avec une date de fin. Rob Walters, par exemple, occupe les clés 4 et 5, la première close le 2009-12-28.

La conséquence pratique est double, et elle vaut des deux côtés. Sur le transactionnel, une récursion mal filtrée contamine tous les niveaux suivants : oublier `enddate IS NULL` dans la partie récursive donne un résultat plausible et faux, ce qui est la pire des combinaisons. Sur l'entrepôt, le même oubli donne un organigramme de 296 personnes là où il y en a 290, et le filtre naïf, lui, donne pire encore : en ne gardant que les lignes courantes des deux côtés de la récursion, on tombe à 254, parce que 23 employés bien courants sont rattachés à une version périmée de leur supérieur. Une hiérarchie historisée se parcourt à une date donnée, pas à coups de filtres sur l'état.

Le dernier écart ne tient ni au dialecte ni au modèle, mais à la nature des deux moteurs, et il est plus brutal qu'on ne l'imagine. Une récursion, c'est beaucoup de tours minuscules : chacun produit quelques lignes, souvent une seule, et le moteur doit relancer sa machinerie à chaque fois. Un moteur transactionnel est taillé pour ça. Un moteur en colonnes est taillé pour l'inverse, balayer beaucoup de lignes en une passe, et le prix fixe qu'il paie par tour devient écrasant quand les tours se comptent par milliers.

La même CTE récursive, celle qui génère les entiers de 1 à N, mesurée sur les deux terrains :

Nombre de tours	PostgreSQL	DuckDB
10 000	4 ms	0,93 s
100 000	42 ms	9,5 s

Deux cent fois plus lent, et l'écart ne se résorbe pas avec la taille : il est proportionnel au nombre de tours, parce que c'est le tour lui-même qui coûte. Voilà l'un des rares endroits de cette formation où la voie transactionnelle l'emporte franchement, et sur la performance, pas seulement sur la fraîcheur.

Deux conséquences pratiques. D'abord, sur l'entrepôt, préférez le raccourci du moteur à la récursion quand il existe : `generate_series(1, 12)` est instantané là où la CTE récursive équivalente fait douze tours pour rien. Ensuite, une récursion sur un entrepôt reste parfaitement acceptable quand elle est courte, et l'organigramme du labo en est l'exemple : cinq niveaux, cinq tours, personne ne verra la différence. Le piège n'est pas la récursion, c'est la récursion profonde.

© 2026 iAlexMG. Tous droits réservés.

Le même travail, deux fois — Le Cours 05 a deux laboratoires, `Labo_05_OLTP_SQL.md` et `Labo_05_OLAP_SQL.md`, qui posent les mêmes questions sur les deux schémas. Faites-les tous les deux et gardez-les ouverts côte à côte. Le labo transactionnel déroule une nomenclature de vélo là où le labo entrepôt déroule un organigramme : deux objets sans rapport, une seule mécanique.

Points clés à retenir

- Une CTE (`WITH`) nomme un résultat intermédiaire valable le temps de la requête. Elle rend le code lisible de haut en bas au lieu de l'imbriquer.
- On chaîne les CTE avec une virgule : chaque étape part de la précédente, comme une chaîne de montage. Pour déboguer, on remplace le `SELECT` final par `SELECT * FROM une_etape`.
- CTE, sous-requête ou vue : la sous-requête est locale et jetable, la CTE structure une requête, la vue est un objet enregistré, réutilisable dans le temps.
- Une CTE récursive a trois morceaux : l'ancre, `UNION ALL`, la partie récursive. Le moteur relance la partie récursive jusqu'à ce qu'elle ne renvoie plus rien.
- Deux usages rois : générer une série (nombres, dates, pour combler les trous) et parcourir une hiérarchie (organigramme ou nomenclature, avec niveau et chemin).
- Portabilité : écrivez toujours `WITH RECURSIVE`, obligatoire sur nos deux moteurs et toléré ailleurs. C'est le cours où le double chemin rapporte le moins d'écarts de dialecte.
- Deux pièges : toujours une condition d'arrêt, car aucun des deux moteurs ne coupe une boucle sans fin ; et la CTE est faite pour la lisibilité, pas pour la vitesse (attention à la matérialisation sur PostgreSQL avant la 12).
- Sur les deux terrains, la syntaxe ne bouge pas. Ce qui bouge, c'est la forme de l'arbre : l'entrepôt vous le sert à plat et sans historique, le transactionnel vous le donne brut, avec ses dates de validité à filtrer vous-même.
- Une récursion profonde coûte cher sur un moteur en colonnes : 100 000 tours prennent 42 ms sur PostgreSQL et 9,5 s sur DuckDB. Sur l'entrepôt, préférez le raccourci du moteur quand il existe ; une récursion courte, elle, ne pose aucun problème.

Exercices

Les exercices de ce cours et leur corrigé vivent dans `Exercices_corriges_05_SQL.md`. Faites-les après les deux laboratoires : plusieurs énoncés supposent que les manipulations ont été faites des deux côtés.

© 2026 iAlexMG. Tous droits réservés.